readcon-core: a reader and writer for the CON checkpoint format

Rohit Goswami[a,1]

[a] *TurtleTech ehf., Reykjavík, Iceland*

Abstract

Rare events in solids and molecules are too slow for a direct trajectory, so a rate calculation searches for saddle points and for the path that joins two minima. The search calls a force evaluator many times, passes geometries between codes, and restarts from a checkpoint. That checkpoint has to carry the cell, a constraint on each Cartesian axis, an atom identity that survives grouping by element, and velocities in a declared time unit. Here we present readcon-core, a Rust reader and writer for the CON checkpoint, with a C interface and bindings for C++, Python, Julia and Fortran. On a band of ten thousand frames the library is about 34 times faster than the ASE reader of the same text. The checks are a conformance corpus, round trips of producer files, and the public HCN to HNC path of a nudged elastic band. The program is released under the MIT licence.

Keywords: CON, checkpoint, nudged elastic band, saddle search, rare events

Program Summary

Program Title: readcon-core
CPC Library link to program files: (to be added by the Technical Editor)
Developer's repository link: `https://github.com/lode-org/readcon-core`
Licensing provisions: MIT
Programming language: Rust for the core, with a C header, a C++ header,

Email address: `rgoswami@ieee.org` (Rohit Goswami)
[1]ORCID: 0000-0002-2393-8056.

and bindings for Python, Julia and Fortran 2008
Supplementary material: the format specification, the conformance corpus, and the benchmark scripts and timing tables
Nature of problem: Harmonic transition state theory estimates a rare-event rate from the barrier and the curvatures at a minimum and at a first-order saddle [1, 2]. Finding those points, and the path that connects two minima, is the work of the nudged elastic band, its climbing-image form, the dimer method, and adaptive kinetic Monte Carlo [3, 4, 5, 6]. A campaign of that kind calls a force code thousands of times, writes a geometry after many of those calls, and starts again from the last good frame. The frame has to remember which Cartesian axes are held fixed, because a slab or a surface is rarely frozen in every direction. It has to remember which atom is which after the file groups atoms by chemical type, because images of a path are compared atom by atom. When the restart continues a trajectory, the velocities have to be in a known time unit, because the kinetic energy scales with the square of that unit. CON is the text checkpoint used for this state. *Solution method:* readcon-core is one parser and one writer for CON text. A frame is a short header, coordinates with a constraint flag and an atom id, and optional sections for velocities, forces and other per-atom fields. Line 2 of a current file is a single JSON object that records the major version, the minor version, the unit system and the section names. From the minor version that makes the time label authoritative, a velocities section is written only with an explicit time unit, and the natural unit shared with existing producers is `(amu*angstrom^2/eV)^0.5`. Decimal numbers are read with correct rounding and, unless the caller asks for a fixed number of digits, written so that they parse back to the same binary64 value. A C header is the narrow waist: C++, Python, Julia and Fortran call that waist, so each language sees one parse. Parallel ingest splits the file on frame boundaries that depend only on the text, and the frames come back in file order at every thread count. gzip and, when the optional feature is built, zstd are compression envelopes around the same text. *Additional comments including restrictions and unusual features:* The C interface reports a failed build or a failed read through a status code and does not keep an error string for a later call. The Python extension is not a free-threaded build. zstd input is accepted only when the library is built with that feature. A pool of threads is used automatically only for files above 48 KiB, and a file that holds a single frame is parsed as one task even if that frame is large. The same fixtures are exercised from the Rust core and from the C, Python, Julia and Fortran bindings. *References:*

[1] G. H. Vineyard, J. Phys. Chem. Solids 3 (1957) 121, doi:10.1016/0022-3697(57)90059-8.
[2] P. Hänggi, P. Talkner, M. Borkovec, Rev. Mod. Phys. 62 (1990) 251, doi:10.1103/RevModPhys.62.251.
[3] H. Jónsson, G. Mills, K. W. Jacobsen, in Classical and Quantum Dynamics in Condensed Phase Simulations, World Scientific, 1998, p. 385, doi:10.1142/9789812839664_0016.
[4] G. Henkelman, B. P. Uberuaga, H. Jónsson, J. Chem. Phys. 113 (2000) 9901, doi:10.1063/1.1329672.
[5] G. Henkelman, H. Jónsson, J. Chem. Phys. 111 (1999) 7010, doi:10.1063/1.480097.
[6] G. Henkelman, H. Jónsson, J. Chem. Phys. 115 (2001) 9657, doi:10.1063/1.1415500.

1. Introduction

Thermally activated events set the rates of diffusion, reactions and structural change, and they are rare on the clock of a direct molecular dynamics run. Harmonic transition state theory estimates such a rate from the barrier height and from the curvatures at the minimum and at a first-order saddle [1, 2]. The practical task is to find those stationary points and the path that joins two known minima. The searches that do that work fall into a double-ended family and a single-ended family. A double-ended search, the nudged elastic band and its climbing-image form, discretizes the path into a chain of images and relaxes that chain until the highest image sits near the saddle [3, 4, 7]. A single-ended search, the dimer method, rotates a pair of replicas until the lowest curvature points uphill and then walks toward the saddle from one basin [5]. Adaptive kinetic Monte Carlo accepts an escape, advances a physical clock by the waiting time of that escape, and repeats the search from the new minimum [6]. Energy-weighted springs, together with eigenvector following, keep a molecular band from cutting the corner of a sharply curved path [8]. An improved estimate of the tangent keeps the elastic band from forming kinks at places where the path bends sharply [9]. The image-dependent pair potential starts the chain of images closer to the minimum energy path the band will converge to [10]. The string method was introduced for the sampling of rare events as a curve that is evolved between two known states [11]. A later form of that method finds a minimum energy path by evolving the curve rather than by relaxing a chain of springs [12]. A comparison of those single-ended searches showed that a saddle can still be found when the product state is unknown [13]. A later dimer method

converges faster than the original rotation because it uses a better estimate of the curvature [14].

These campaigns spend most of their effort on force calls, and they spend the rest on moving the current geometry from one program to another and on starting again after a stop. A reproducible band optimization calls a potential for every image at every iteration, and the checkpoint is what lets that run resume [15]. A tutorial review of Gaussian-process searches shows how a surrogate can cut the number of force evaluations on a stationary-point problem [16]. The same idea has been implemented for molecular saddle searches, where the surrogate still has to land on a force evaluator [17]. Adaptive pruning then drops training points that no longer change the next step, which keeps a long search from storing every geometry [18]. An earlier Gaussian-process band uses inverse interatomic distances so that a nudged elastic band needs fewer calls to the potential [19]. A Gaussian-process minimum-mode search uses the same kind of covariance to rotate a dimer toward a saddle from one basin [20]. Eigenmode alignment of the climbing image keeps the highest bead on the softest mode of the Hessian while the band relaxes [21]. The same files are read again when a path is projected onto two dimensions and compared with the band that produced it [22]. A long kinetic Monte Carlo run writes a checkpoint whenever it accepts a move, and that file has to be readable after the job has stopped [6]. Between those reads the file may sit on disk for hours while a queue waits, or it may be copied onto another machine that has a different build of the force code. The older eOn paper is the reference for the long-timescale code in which this checkpoint became the working file, and a restart is only as faithful as the frame that code wrote [23].

The checkpoint has to carry three facts that a bare list of coordinates does not. A slab or a surface calculation holds some Cartesian axes fixed and leaves others free. The constraint is therefore a mask with one bit for x, one for y and one for z, rather than a single flag for the whole atom. A platinum slab makes the distinction concrete. A buried atom that must not move carries the integer 7, because bits 0, 1 and 2 are all set, and a surface atom that must relax carries 0. An adsorbate that may travel in the plane while its height is held carries 4, which sets only the z bit, and that integer is a different physical constraint from either 0 or 7. An atom identity has to survive the moment the file groups atoms by chemical type. Image i of a band must still refer to the same atom as image $i+1$, and the two endpoints of a reaction must still align before a distance is computed. Velocities, when

the restart continues a trajectory, have to be paired with a time unit, so that the kinetic energy of the restarted system is the kinetic energy that was stored. A campaign that drops any one of those three facts can still produce a plot, and the stationary point it reports will not be the stationary point the constraints defined.

CON is the text form of that checkpoint, written so that a failed search can be inspected without a decoder. It grew up inside eOn, which writes one frame per minimum, saddle or band image and reads it back on the next cycle [23]. The Atomic Simulation Environment reads and writes the same text, which is how a geometry moves from the saddle search into a Python workflow [24]. A band file is therefore a sequence of frames that several programs already agree to exchange, and the agreement is only useful if each program keeps the same columns.

General trajectory formats solve a neighbouring problem and leave this contract open. H5MD stores positions, optional velocities and a time unit as attributes on HDF5 datasets, and it does not define a per-axis constraint or a pre-grouping identity [25]. NetCDF is an array container, and any checkpoint convention built on it has to be agreed outside the container itself [26]. The TNG format stores coordinates, velocities, forces and a box for molecular simulation, with atom identity taken from the topology rather than from a column that survives reordering inside the frame [27]. Extended XYZ, as used through ASE, can add columns and a comment line, and it does not reserve a three-bit mask or a versioned time unit for this checkpoint [24]. A LAMMPS dump can carry an atom id and, when the dump style asks for them, velocities and forces [28]. The unit style and the fixed axes live in the input of the run rather than in the dump frame. The LAMMPS data file holds the starting geometry, the dump holds the trajectory, and neither of those files is the CON band of images [28]. H5MD is stored in HDF5, and that container is what holds the arrays of positions and of optional velocities [29]. Zarr is a chunked array store for the same kind of grid, and its community standard is what defines the layout of those chunks [30]. CHARMM writes a DCD trajectory of coordinates and of the box, which is a record of a dynamics run rather than of a saddle search [31]. GROMACS writes XTC and TRR trajectories, and a reader of those files gets positions, and sometimes velocities, without this checkpoint's mask [32]. MDTraj reads DCD and XTC trajectories when the task is analysis of a finished run, which is a different job from restarting a search [33]. A protein structure is often stored as a PDB file, and atom identity there comes from the residue and the atom name [34]. mmCIF is

the macromolecular archive form of that structure, and it does not carry this checkpoint's mask or its time unit [35]. A small-molecule crystal structure travels as a CIF, which archives the cell and the coordinates but not a velocity time unit for a restart [36]. A plane-wave calculation reads a POSCAR geometry, the cell and the coordinates for VASP, and that file is not a band of images with masks [37]. pymatgen converts among the crystal files used in solid-state work, including POSCAR and CIF, and the result is still a geometry [38]. Atomsk converts among the files used for defects and for classical potentials, which is how a geometry changes dialect before a run [39]. Open Babel converts among the chemical formats used for molecules, and those formats name bonds and residues rather than a per-axis mask [40]. OVITO displays atomistic snapshots and can read several of these files, and the display is not the checkpoint a saddle search restarts from [41]. i-PI drives a calculation by passing coordinates to a client and reading forces back, which moves a geometry without storing this checkpoint [42]. That coordinate exchange began as a path-integral client, which sent positions out and received forces back, one geometry at a time [43]. A liquid or a mixture is often packed into a cell before the first minimization, and that packed geometry is what the checkpoint then stores [44]. Machine-learning potentials exchange atomic features as tensors, and a tensor of features is a different object from the geometry checkpoint a search writes [45]. Table 1 puts those differences in one place, after the file itself has been described.

A campaign that restarts from a lossy reading of this checkpoint loses the calculation in ways that change the answer. A per-axis constraint that has been stored as a single boolean either freezes an atom that should still move in the plane or frees an atom that should have stayed put. The next force call then follows the wrong surface. On the platinum slab that mistake turns the adsorbate with mask 4 into a rigid spectator, so the search never samples the diffusion path the setup was written to allow. An identity that was dropped when atoms were grouped by element makes two images of a band impossible to compare. The path can then be plotted only after an extra alignment that the file was supposed to make unnecessary. A velocity block whose numbers are in one time unit and whose label names another multiplies the kinetic energy by the square of that mistake. A thermostat or a kinetic Monte Carlo acceptance then sees a different temperature from the one that was saved. The damage shows up later, as a rate, a barrier or a temperature, rather than as a message at the moment the file was read.

Here we present readcon-core, a reader and writer for CON whose job is to

keep that contract. The library is a Rust parser and writer, exposed through a C header so that C++, Python, Julia and Fortran share one reading of the file. What is new is the versioned units rule for velocities, the round-trip default for floating-point text, and the preservation of unknown metadata keys. The parallel ingest is new in the same sense: its frame boundaries do not depend on the number of threads. The same program also accepts the older files that eOn and ASE have been writing, including velocities stored in the natural unit. Section 2 explains the design of the checkpoint, and Section 3 describes the library and the bindings. Section 4 reports the conformance checks, the unit example and the HCN to HNC path. Section 5 reports paired timings and thread scaling, and Section 6 states the limits of the program as released. Section 7 collects the result.

2. The CON format

A CON file is a sequence of frames in plain text, so that a failed search can be inspected without a decoder, and Figure 1 shows how one frame is laid out. The first nine lines are the header. They carry a generator note, a metadata line, the three cell lengths and the three cell angles, then two further text lines. The last three of those nine lines are the number of chemical types, the count of atoms of each type, and the mass of each type. Coordinates follow, grouped by type, and each group is a chemical symbol, a line that names the component, and one row per atom. A row holds x, y, z, a constraint flag and an atom id, five tokens on one line, and the parser treats a missing token or an extra token as a broken row rather than as a default. Optional sections, velocities first when they are present, repeat that grouping after a blank line. Lengths in the files discussed here are in angstrom, $1\,\text{Å} = 10^{-10}\,\text{m}$, and energies are in electronvolts. Walking one coordinate line is enough to see the contract: the first three tokens are the Cartesian position, the fourth is the mask described below, and the fifth is the identity that has to survive the grouping by element.

The cell is stored as three lengths and three angles in degrees, which is enough to rebuild the box once a convention for the angles is fixed. The angle on line 4 between b and c is α, the angle between a and c is β, and the angle between a and b is γ. The convention puts a along x and b in the xy plane, which is how both eOn and ASE turn those six numbers back into vectors [23, 24]. When the metadata also carries lattice vectors, those vectors are three rows of the cell matrix and they take priority on read. The writer then

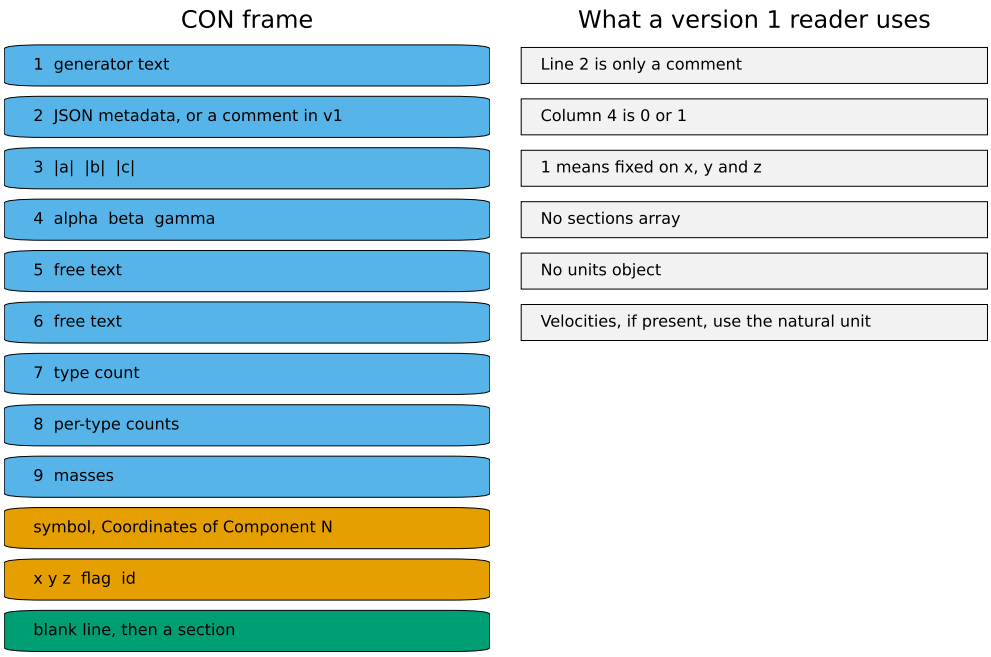


Figure 1: Left: the lines of one CON frame, from the generator note through an optional section. Right: the subset a version 1 reader uses. A version 1 file has a comment on line 2, treats column 4 as 0 or 1, and has no units object. Blue marks the header, orange the coordinate group, and green the section that follows a blank line.

keeps lines 3 and 4 consistent with the vectors at the precision it is using. A missing periodicity flag means the cell is periodic in all three directions, which is the usual case for a bulk crystal and for a slab that is periodic in the plane. A caller who needs a non-periodic direction sets the flag explicitly, because silence is the periodic default and not an invitation to guess.

Column 4 is the constraint, stored as an integer token so that a later reader does not have to recover axes from a Boolean. In a version 1 file the value 0 leaves the atom free and the value 1 fixes x, y and z together, which matches the files those early producers wrote. From version 2 onward the same column is a bitmask: bit 0 fixes x, bit 1 fixes y, and bit 2 fixes z, so the legal values run from 0 to 7. A slab with a frozen substrate and a free surface is the case this bitmask is for, and the platinum example in the introduction is exactly the integers 7, 0 and 4. Fixing only z on an adsorbate, or only x on an atom that should still relax in the other directions, is a different

physical constraint from fixing the atom outright. The file has to be able to say which constraint was meant. A decimal spelling, a minus sign or a value above 7 is rejected in every version, because a flag that is not an integer mask cannot be turned into axes without a guess. The rejection is the point of the check. A killed job that wrote a truncated row, or a converter that printed a floating-point flag, stops at that row. The rest of the campaign does not relax the wrong axes from a row the parser refused.

Column 5 is the atom id, a non-negative integer token read as an integer up to $2^{64} - 1$, and not through a floating-point conversion that would round large values. The reader does not assume that ids start at 0 or at 1, writers emit ids that are unique inside the frame, and readers still accept a repeated id. A repeated id keeps the order of the rows in the file, which is the rule eOn uses when a file was produced with duplicates [23]. When the ids are unique, sorting them recovers the order the atoms had before they were grouped by type, which is the order a distance or a projection expects. A coordinate line that has only four columns, and therefore no id, takes the zero-based position of that atom in the frame, so an older file remains readable. The id is what lets a band image be compared with the next image after both files have grouped carbon with carbon and hydrogen with hydrogen. Without that column the grouping is a silent permutation, and two frames that describe the same reaction can no longer be zipped atom by atom.

Line 2 is one line of UTF-8 JSON, without a byte-order mark. The major version is a JSON number under the key for the specification version. The minor version is a second integer. A reader accepts an unknown minor of a major it knows, and it refuses an unknown major, so a later clarification can travel in a file that an older program still opens. JSON that claims major version 0 or 1 is rejected, because those generations were not JSON. Unknown keys on the line are preserved when the frame is written again, so a campaign can attach an energy, a bead index or a potential name without waiting for every reader to learn the key. A section name that the reader does not know is an error, unless the file’s minor version is newer than the reader and the metadata declares how many numbers that section carries per atom. In that case the reader may skip the block. New reserved names therefore arrive with a minor bump, and a reader of the same major can still open the file. Line endings may be LF or CRLF. Trailing spaces and tabs are ignored, and labels such as the coordinate heading are compared without regard to case after runs of spaces have been collapsed.

Units live in the metadata, length and energy are required once the file

is major version 3, and mass is recorded as well. The time unit is the one that decides the meaning of a velocity, because a velocity component is a length divided by a time and the kinetic energy squares that scale. The natural unit used by eOn and by ASE is $(\text{amu}\cdot\text{angstrom}^2/\text{eV})^{0.5}$, which equals 10.1805055 fs in the eOn clock, and $1\,\text{fs} = 10^{-15}\,\text{s}$. The library keeps that spelling when it writes the unit, rather than rewriting it with spaces or with a different algebraically equal form, so a later textual comparison still matches. Version 1 and version 2 files have no units object, and their velocities, when a velocity block is present, are in the natural unit, because that is the unit those producers have written. A version 3 file that has no minor number is treated the same way: the time label is not trusted, and the numbers are read in the natural unit. The producers of those velocity sections wrote natural-unit components and labelled the time in femtoseconds, so trusting the label would scale every component by 10.1805055 and the kinetic energy by 103.64. From the minor version that records time, the label is authoritative, and a writer that emits velocities must emit the time unit with them. A writer that is given a unit it cannot parse fails, rather than substituting a default, and the caller sees the failure at the moment the frame is built.

Sections are named blocks after the coordinates, and the reserved name for per-atom energies is energies. A velocities section, a forces section, and further blocks for charges, spins, magnetic moments, displacements and spreads follow the same pattern as the coordinates: a symbol, a heading, and one row per atom. Exactly one blank line stands before each section, and no blank line stands between the component blocks inside a section, which is how the reader knows that a new name is a new section rather than a new element. The metadata lists the names that are present, so a reader knows what to expect before it reaches the first row, and a listed name that never arrives is an incomplete frame. A count of zero types is a legal empty frame: the two lines after the count are blank, no coordinate block follows, and the section list is empty. Writers aimed at programs older than this minor version should avoid that empty frame, because those programs may not accept a header that declares no atoms. Trailing whitespace after the last complete frame is ignored, while a header that stops early, or a section with fewer rows than the header declared, is an error, and the partial frame is not returned as if it were complete. A campaign that was killed while writing therefore fails the next read at the truncated frame, instead of continuing from a geometry that was never finished.

The numeric contract is stated in round-trip terms, which is the property

a checkpoint needs if the same geometry will be parsed more than once. Readers convert decimal text to binary64 with correct rounding [46, 47]. The default writer prints each finite value so that reading it back yields the same binary64 number, which is the shortest-round-trip problem solved by Steele and White [48]. Grisu prints a floating-point value using only integer arithmetic so that the decimal form is short and correctly rounded [49]. Ryu prints a floating-point value by a table-driven procedure so that the decimal form is short and correctly rounded [50]. The metadata line is JSON, and a scanner for that kind of text has been shown to run at a very high rate on such input [51]. A caller may instead ask for a fixed number of digits after the decimal point. That mode is allowed, and it is lossy: values that differed in the last bits can collide, and values that needed more digits no longer parse back to themselves. Non-finite numbers, including the spellings of NaN and infinity, are rejected in every field and in every mode, so a diverged force call cannot be stored as a coordinate and read back as a number. The specification distributed with the program is the normative text for these rules. Compression does not change that text: gzip and zstd are envelopes around the same bytes, with the usual extensions for a CON file and for a velocity file. A gzip file may contain more than one member, and every member is read, in order, which matters when a concatenated archive holds one frame per member. zstd support is present when the library is built with that feature, and the bytes that come out of either envelope are parsed by the same reader as an uncompressed file.

3. Implementation

The core is a Rust library with a single parser, and file reads, string reads and the bindings all go through that parser, so a change in the grammar shows up once. The parser walks the header, checks the counts, reads each coordinate row as an integer flag and an integer id, and then reads the sections named in the metadata. Decimal fields go through a fast correctly rounded conversion [47]. The public entry points a caller needs are few. read_all_frames reads every frame from a path. A frame builder assembles a frame from a cell, atoms, masks and optional sections, and the writer turns that frame back into text. Keeping one parser is what makes the later conformance story possible, because a second implementation in each language would be a second place for a mask or a time unit to drift.

The C header is the waist of the library: it owns the frame on the far side

of the call, returns a status code, and expects the caller to free what it was given. C++ is a header over that interface, Julia and Fortran call the same symbols, and the Fortran side uses the standard C binding. Python uses the same parse, so a file that is legal in one binding is legal in the others, and a file that is rejected is rejected for the same reason. The status code is the whole of the error report on the C side, and there is no stored string that a later call can pick up, which is a limit Section 6 states again. That narrow report is a deliberate shape for the waist: every language maps one integer, and none of them invents a second message for the same failure.

The frame builder is where a program constructs a checkpoint rather than parsing one. The caller sets the cell, the symbols, the coordinates, the mask and the id, and any velocities or forces it has. On a version 3 frame the builder writes the minor number when the caller did not set one, and it writes a units object when the caller did not supply one. It does not invent a femtosecond label for velocities, because that label is the historical mistake the units rule exists to stop. If the frame has a velocities section and the minor version requires a time unit, the build fails unless that unit was set. When an older frame that already contains velocities is rewritten, the natural unit is recorded on the new metadata line, so the next reader does not have to guess which clock the components used. The default text form of every floating-point field is the round-trip form, and fixed digits remain a separate request for a caller who wants a shorter file and accepts the loss.

Parallel ingest is a two-step read, which matches the way a band is already a list of frames rather than one large array of atoms. A serial scan finds the byte offset where each frame starts, using the header counts so that it can skip a frame without building its atoms, and the pool then parses each slice. The offsets depend on the bytes of the file and not on how many workers are waiting, and the frames are assembled in file order. The automatic policy stays on the serial parser for a buffer below 48 KiB, where the cost of starting the pool is larger than the parse, and it uses the pool above that size. A file with one frame is one task, however many atoms that frame contains, because the split is on frames and not on atoms. A fat single image does not become faster just because the machine has idle cores. read_all_frames takes an optional thread count: one thread forces the serial path, and two or more pin a pool of that size. The result is the same sequence of frames at every count, which Section 5 checks by hashing every stored field, including the mask, the id, the velocities and the metadata. A search that farms images out to workers can therefore choose a thread count for throughput, and the

frame it hands to the force code does not depend on that choice.

The Python surface is the one most campaigns will call, and read_all_frames and read_con both return frames. from_ase builds a frame from an ASE Atoms object: positions, cell lengths and angles, masses, a fully fixed atom where ASE stored that constraint, a per-axis mask where ASE stored a Cartesian constraint, and velocities when ASE has them. The numbers are copied as ASE holds them, so a caller who has velocities in a known unit sets that unit on the builder, and the library does not convert angstrom per femtosecond into the natural unit behind the caller's back. The reverse direction writes frames back out for ASE. chemfiles is an optional ingress for formats other than CON, and when the library is built with it, read_chemfiles opens an XYZ file or another format chemfiles supports and returns frames through the same data structures [52]. The CON text remains the file that eOn and ASE have to read, and other formats are a way in rather than a second checkpoint. The Python call releases the interpreter lock while the file is read and parsed, and it takes the lock back to build the Python objects that the caller sees. Other Python work can therefore run during a long parse, and the frames that come back are still in file order.

A campaign touches the same frame more than once, which is why the rules above are checked at each touch. The builder records the cell, the mask and the id before the first force call, and the writer emits the text a queue will store. A later rank reads that text, compares the ids, and calls the potential on the free axes only. If that rank writes the frame again, unknown keys such as a bead index or a stored energy are still on the metadata line. The round-trip printer is what keeps the coordinates still on that second write, because a short fixed-digit print would move atoms by a last-bit rounding on every restart. The units rule is what keeps a velocity restart honest, because the kinetic energy the next step sees is the energy the previous rank stored. The file those steps produce is the one the existing searches already pass from rank to rank.

4. Validation

The conformance corpus is a set of small files, some legal and some not. The fixtures cover the header, the constraint mask in both the version 1 sense and the bitmask sense, a frame with no atoms, the units line, and files that must be rejected. The same fixtures are wired from the Rust core and from the C, Python, Julia and Fortran bindings, so a binding cannot

quietly accept a file the core rejects. Table 1 places the CON contract next to the neighbouring formats cited above. The CON column is what this library keeps, and the ASE column is what its eon reader keeps. Section 4.2 measures that ASE column on a real path. A legal fixture that one binding dropped, or an illegal fixture that one binding accepted, would show up as a different status on that binding, which is why the fixtures are shared rather than rewritten per language.

Table 1: What a restarted rare-event calculation needs, and what each file actually stores. The CON column is what readcon-core keeps.

Item	CON	ASE eon	ext. XYZ	H5MD	LAMMPS du
mask	per-axis bits	one boolean	no standard mask	not specified	not in the du
id	kept	not kept	optional column	optional	atom id
velocity unit	declared	not read	not required	time attribute	outside the fi
sections	named blocks	coordinates only	property columns	particle groups	optional colu
version	major and minor	not read	none	format version	none

Reading Table 1 row by row shows where a restart would lose the calculation. The mask row is the sharpest split. CON keeps one bit for each Cartesian axis, while the ASE eon reader keeps a single boolean, and extended XYZ has no reserved mask. H5MD does not specify a per-axis constraint, and a LAMMPS dump leaves the fixed axes in the input of the run rather than in the frame. The identity row is the next place a path tool fails. CON keeps the pre-grouping id, and the ASE eon reader does not keep it. The other three formats treat an atom id as an optional column or as an index that belongs to a topology. A band that has been grouped by element can be compared image to image only when that id comes back with the coordinates. The velocity row is the kinetic-energy hazard taken up in Section 4.1. CON declares the time unit on the metadata line, and the ASE eon reader does not read a velocity section. Extended XYZ does not require a time unit, while H5MD can store time as an attribute on the dataset. That attribute is the right idea for a molecular trajectory, and it is a different file from this checkpoint. A LAMMPS dump leaves the unit style outside the frame, so a reader of the dump alone cannot recover the clock the velocities used. The section row records the same kind of split. CON carries named blocks for velocities, forces and the other per-atom fields, and the ASE eon reader stops at the coordinates. Extended XYZ can add property columns, H5MD uses

particle groups, and a LAMMPS dump can request extra columns. None of those choices is the named block this checkpoint already uses between eOn and ASE. The version row is what lets a later minor travel in a file an older reader of the same major still opens. CON stores a major and a minor, and the ASE eon reader does not read the JSON line. Extended XYZ and a LAMMPS dump have no such version, and the H5MD format version belongs to the container rather than to this mask. Taken together, the table maps what a restarted saddle search needs onto the file that already stores it. Section 4.2 then checks the two columns a working campaign actually meets, CON through this library and the same text through the ASE eon reader.

Files written by the library are read back by the same library. In the default mode every finite value parses back to the binary64 number that was stored, which is the check behind the round-trip writer. The public endpoints used below were stored as CON text, rewritten with a current metadata line, and read again, and the benchmark files in Section 5 are the same kind of text, repeated or tiled. Both readcon-core and the ASE eon reader are asked to open those benchmark files, so the timing comparison is a comparison of two readings of one checkpoint rather than of two unrelated containers.

4.1. A time unit and a kinetic energy

The natural time unit and the femtosecond differ by a factor 10.1805055, and kinetic energy is quadratic in velocity, so reading natural-unit components as if they were angstrom per femtosecond multiplies the energy by 103.64. On a 336-atom platinum slab at 300 K the kinetic energy in the natural unit is 12.386 eV, and the same numbers, read as angstrom per femtosecond, give 1283.7 eV. A restart that makes that mistake continues a much hotter trajectory than the one that was saved, and a thermostat or an acceptance step then acts on the wrong temperature. From the minor version that stores the time unit, the label prevents the mistake, and on older files the reader uses the natural unit, which is the unit the producers wrote. Squaring that scale is required because the kinetic energy depends on the square of each velocity component, which is why the two energies above differ by 103.64.

4.2. HCN to HNC

The public HCN and HNC endpoints of an eOn nudged elastic band are the geometries used to illustrate a reproducible path optimization [15]. Those endpoints are rewritten here as current CON, with the natural time

unit declared and a velocities section added. A third frame keeps the HCN coordinates, sets the hydrogen flag to the x-only bit, and stores the ids 2, 0 and 1 in file order. The file still groups the atoms as carbon, then nitrogen, then hydrogen, so the third row is hydrogen even though its id is 1 and the carbon in the first row carries id 2. A caller who zips the rows in the order they appear will pair the wrong atoms once those ids have been permuted, and a caller who sorts on the id column recovers the order the path tool expects. Table 2 lists what readcon-core and the ASE eon reader return. readcon-core returns the three-bit mask, the id and the velocity components, and it records the time unit that was written. The ASE eon reader fails on the file that contains the velocities section, so that read returns neither the id nor the velocity components. On the coordinate form of the probe, which has no section, a nonzero flag becomes a fully fixed atom, so the x-only hydrogen is reported as fixed on every axis.

Table 2: Mask, atom id and velocity returned for the rewritten HCN and HNC endpoints. A mask is three bits in order xyz. The probe frame fixes hydrogen on x only and stores ids 2, 0 and 1.

Frame	Reader	Mask	Id	Velocity
HCN	readcon-core	000 000 000	0 1 2	present
HNC	readcon-core	000 000 000	0 1 2	present
probe	readcon-core	000 000 100	2 0 1	present
sectioned file	ASE eon	failure		
probe, coordinates	ASE eon	free free all-fixed	absent	absent

A path tool that aligns these endpoints by the id column can do so from the readcon-core frame, because the ids are the ones that were written, including the permutation 2, 0, 1 on the probe. A tool that trusts the ASE eon reader has to invent an identity after the sectioned file fails to parse, and on the coordinate form it cannot tell an x-only constraint from a fully fixed atom. The velocities are present so that a restarted frame has a kinetic energy in the declared time unit, and they are present only on the reader that keeps the section. The three-atom path is small on purpose: if the mask, the id and the time unit survive here, they are the same fields a slab of hundreds of atoms asks the reader to keep.

5. Performance

We compare readcon-core with four peers on five payloads, on one Intel Xeon Processor with 8 hardware threads. The peers are the ASE eon reader on the same CON text and a chemfiles read of an XYZ file with the same coordinates [24, 52]. The other two peers are an MDAnalysis read of an H5MD file with the same coordinates, and an h5py read of that file's position array [53, 54]. The payloads are 1, 100 and 10 000 frames of a 218-atom geometry, one frame of 10 000 atoms, and a short band file that carries sections. Each cell is 30 paired repeats, interleaved so that a slow drift in the machine hits both sides of the pair, and the ratio is the readcon-core time divided by the peer time. We report the median and a bias-corrected 95 percent interval, and we apply a Holm adjustment across the cells that produced a ratio, while timeouts and reader failures stay in the table. The paired baseline leaves the thread count unset, so the automatic policy uses the pool once the file is above 48 KiB, and each peer runs as its own single process. The single frame of 10 000 atoms stays one task under that policy, because the split is on frames. The full protocol and the rounded ratios are in Appendix A, and Figure 2 shows the intervals.

Against the ASE eon reader the speedup, the reciprocal of the ratio, runs from 11 to 34 on the files both programs finish. Against chemfiles on XYZ the factor runs from 2.4 to 6.1. Against MDAnalysis on H5MD it runs from 5 to 95. The case a user actually waits on is the band of ten thousand frames. There readcon-core is 34 times faster than ASE, 6.1 times faster than chemfiles, and 55 times faster than MDAnalysis. h5py is faster on that band: the readcon-core time is 5.7 times the h5py time, which is the expected direction when the peer reads one dense array and readcon-core parses text. On a single small frame the comparison with h5py goes the other way, and readcon-core is 6.3 times faster. The single frame of ten thousand atoms is parsed as one task. There readcon-core is 11 times faster than ASE, and h5py is again faster, by a ratio 2.7. The sectioned band is a failure for every peer, 30 times out of 30 for the ASE reader, and those rows are kept. The peers are not CON section readers, and the timing table does not turn that failure into a speedup.

Figure 3 pins the pool at 1, 2, 4 and 8 threads on the ten thousand frame file. Two threads give a speedup of 2.4, which sits above the ideal line, and four and eight threads give speedups of 3.5 and 4.7. The boundary scan that finds the frame offsets is serial, and each frame is a small parse, so the extra

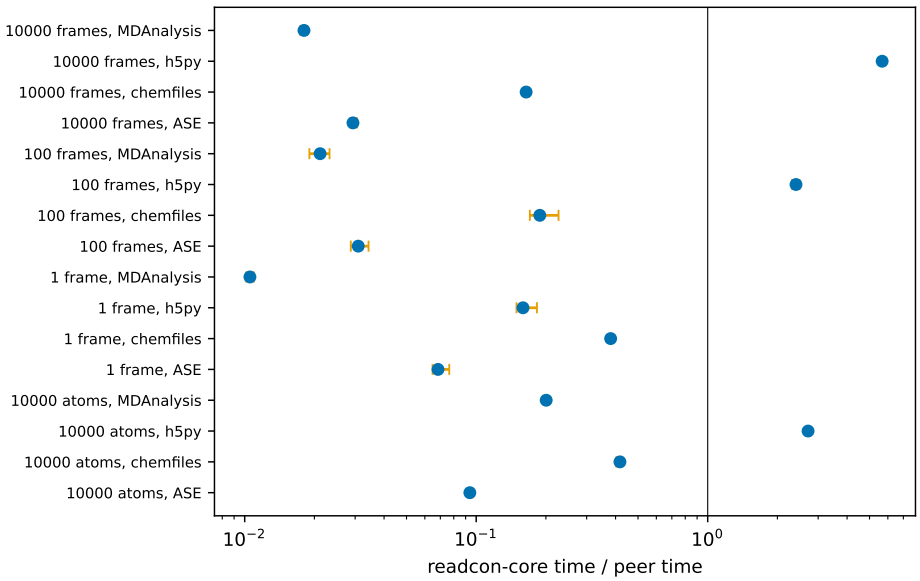


Figure 2: Median ratio of readcon-core time to peer time on the paired reads. A ratio below 1 means readcon-core finished sooner. The bars are bias-corrected 95 percent intervals of the median. The vertical line is a ratio of 1. The ten thousand frame band and the ten thousand atom slab are the large cases.

workers at eight threads share a shrinking amount of independent work. The parsed frames do not depend on that choice. The field hash is the same at 1, 2, 3, 4 and 8 threads.A campaign can therefore use the extra cores for a long band, and the frames that the next code reads stay the ones that were stored.

6. Limitations

Two limits are part of the program as released, and both of them are visible from the interface rather than from a missing file format. The C interface does not store an error string, so a caller that receives a failing status has the status and not a sentence to show the user. The Python extension is not a free-threaded build, so a caller who shares frames across threads keeps the usual locking around the extension. zstd envelopes need the optional build, and a library built without that feature rejects a zstd input instead of silently treating it as text. A single large frame does not split across threads, because the parallel work is the set of frames and not

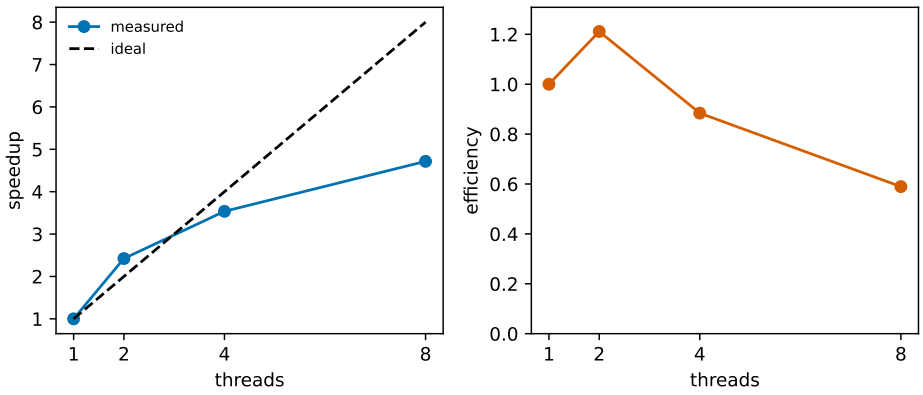


Figure 3: Thread scaling on the ten thousand frame file. Left: speedup against the one-thread median, with the ideal line drawn dashed. Right: parallel efficiency. The field hash of every stored value is the same at 1, 2, 3, 4 and 8 threads.

the set of atoms, which is the same rule the ten thousand atom timing uses. Fixed-digit output remains available for a caller who wants a short file, and it is lossy next to the round-trip default, so a geometry that must come back unchanged should stay on the default writer.

7. Conclusions

Rare-event campaigns restart from a checkpoint that has to remember per-axis constraints, atom identity and the time unit of any velocities. CON is that checkpoint in the eOn and ASE workflow, and readcon-core is a reader and writer for it, with one parser behind a C header and bindings for C++, Python, Julia and Fortran. The versioned units rule keeps a restarted kinetic energy equal to the stored one, which on the 336-atom platinum slab is the difference between 12.386 eV and 1283.7 eV. The HCN to HNC endpoints show the same split on a published path. The library returns the mask, the id and the velocities. The ASE eon reader fails on the sectioned file, and it collapses a nonzero flag when the section is absent. On the ten thousand frame band the library is 34 times faster than that ASE reader, while h5py remains faster at reading a dense array of positions. The frames that come back do not depend on the number of threads, so a long band can use the extra cores without changing the geometry the force code will see.

Data availability

The software is archived at the Zenodo concept DOI 10.5281/zenodo.18753567 and will be deposited in the CPC Program Library [55]. A numbered release of the library is archived beside the concept record, so one version can be cited as well as the series [56]. The archive is the software object that the FAIR principles for research software ask an author to identify and to describe [57]. A citation to research software is owed to the archived record, which is why the concept DOI appears in this paper [58]. The benchmark scripts and the per-repeat timing tables are part of the submission archive. The HCN and HNC endpoints are the public files of the nudged elastic band study [15]. The public endpoints are written by the orchestrator that drives the band, and the file it stores is the checkpoint this library reads [59].

CRediT authorship contribution statement

Rohit Goswami: Conceptualization, Methodology, Software, Validation, Visualization, Writing - original draft, Writing - review & editing.

Declaration of competing interest

The author declares that he has no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This work received no funding.

Acknowledgements

Hannes Jónsson and Graeme Henkelman built the saddle-search practice that this checkpoint has to survive, and Samuel T. Chill led the eOn paper that made CON the file that practice writes. Miha Gunde's work on eigenmode-aligned bands made a stable atom identity part of the path, not an afterthought. Ruhila Goswami chose the colours used in the figures. The author's family stayed with the writing of the library, and so did the pets, plants, birds, garden cats and turtles.

Appendix A. Benchmark protocol and ratios

The host label in the timing tables is bench-vm. The processor, the hardware-thread count, the compiler, the library revision, the script revision and the UTC date are columns of the per-repeat table. Payloads are built from one 218-atom frame. The 1-frame, 100-frame and 10 000-frame files repeat that frame. The 10 000-atom file tiles one coordinate row out to 10 000 atoms in a single frame. The sectioned file is a four-image band with extra sections. XYZ and H5MD peers receive coordinates taken from the same frames. The sectioned file is handed to the peers as it stands, and a peer that cannot parse it is a failure. For each payload and each peer the loop is baseline, peer, baseline, peer, thirty times. A repeat that exceeds 240 seconds is a timeout. A peer that raises is a failure. Neither is dropped, and neither enters the median. The ratio uses only pairs in which both sides returned a time. The interval is a bias-corrected and accelerated bootstrap of the median, with 9999 draws. A two-sided bootstrap probability for the median ratio equalling 1 is adjusted by the Holm step-down across those cells. Figure 2 draws the cells that have an interval. Table A.3 lists every cell, including the failures. Thread timings use the 10 000-frame file at 1, 2, 4 and 8 threads, thirty repeats each, and the field hash is also taken at 3 threads so that the identity check covers 1, 2, 3, 4 and 8.

Appendix B. An indexed corpus of CON text

readcon-db stores CON text in an LMDB file and indexes it. The text stays the authority. The indexes answer the questions a campaign asks before it parses a frame: which entries have a given composition, which energies fall in a range, and which frames carry forces or velocities. An exact-match hash of the stored text finds duplicates. The library is a layer on top of readcon-core, and the note below is only the index a search needs before it opens a frame. Opening a corpus, appending a band and selecting on copper plus an energy window looks like this.

```
use readcon_db::{ConCorpus, Select};
let db = ConCorpus::open("corpus")?;
db.append_trajectory_path(1, "band.con")?;
let keys = db.select(
    &Select::new()
        .require_symbol("Cu")
```

Table A.3: Paired time ratios. The ratio is the readcon-core time divided by the peer time, so a value below 1 means readcon-core finished sooner. The interval is a bias-corrected and accelerated 95 percent interval of the median. Failures stay in the table and are not converted into times.

Payload	Peer	OK	Fail	Ratio	Interval
10000 atoms	ASE eon	30	0	0.0939	[0.0921, 0.0958]
10000 atoms	chemfiles XYZ	30	0	0.418	[0.403, 0.427]
10000 atoms	h5py	30	0	2.71	[2.68, 2.74]
10000 atoms	MDAnalysis H5MD	30	0	0.201	[0.197, 0.203]
1 frame	ASE eon	30	0	0.0684	[0.0648, 0.0765]
1 frame	chemfiles XYZ	30	0	0.381	[0.378, 0.384]
1 frame	h5py	30	0	0.159	[0.149, 0.183]
1 frame	MDAnalysis H5MD	30	0	0.0105	[0.0102, 0.0109]
100 frames	ASE eon	30	0	0.0309	[0.0287, 0.0343]
100 frames	chemfiles XYZ	30	0	0.188	[0.171, 0.227]
100 frames	h5py	30	0	2.41	[2.31, 2.49]
100 frames	MDAnalysis H5MD	30	0	0.0212	[0.019, 0.0233]
10000 frames	ASE eon	30	0	0.0294	[0.0285, 0.0306]
10000 frames	chemfiles XYZ	30	0	0.164	[0.161, 0.168]
10000 frames	h5py	30	0	5.67	[5.59, 5.75]
10000 frames	MDAnalysis H5MD	30	0	0.018	[0.0174, 0.0186]
sectioned band	ASE eon	0	30		
sectioned band	chemfiles XYZ	0	30		
sectioned band	h5py	0	30		
sectioned band	MDAnalysis H5MD	0	30		

```
        .energy_range(-50.0, 0.0),
)?;
let digest = db.frame_hash(keys[0])?;
```

The selection returns keys. The hash returns the digest of the text stored under a key. Decoding a hit uses the same reader as the rest of this paper.

References

[1] G. H. Vineyard, Frequency factors and isotope effects in solid state rate processes, Journal of Physics and Chemistry of Solids 3 (1) (1957) 121–127. doi:10.1016/0022-3697(57)90059-8.

[2] P. Hänggi, P. Talkner, M. Borkovec, Reaction-rate theory: fifty years after Kramers, Reviews of Modern Physics 62 (2) (1990) 251–341. doi:10.1103/RevModPhys.62.251.

[3] H. Jónsson, G. Mills, K. W. Jacobsen, Nudged elastic band method for finding minimum energy paths of transitions, in: B. J. Berne, G. Ciccotti, D. F. Coker (Eds.), Classical and Quantum Dynamics in Condensed Phase Simulations, World Scientific, 1998, pp. 385–404. doi:10.1142/9789812839664_0016.

[4] G. Henkelman, B. P. Uberuaga, H. Jónsson, A climbing image nudged elastic band method for finding saddle points and minimum energy paths, The Journal of Chemical Physics 113 (22) (2000) 9901–9904. doi:10.1063/1.1329672.

[5] G. Henkelman, H. Jónsson, A dimer method for finding saddle points on high dimensional potential surfaces using only first derivatives, The Journal of Chemical Physics 111 (15) (1999) 7010–7022. doi:10.1063/1.480097.

[6] G. Henkelman, H. Jónsson, Long time scale kinetic Monte Carlo simulations without lattice approximation, The Journal of Chemical Physics 115 (21) (2001) 9657–9666. doi:10.1063/1.1415500.

[7] D. Sheppard, R. Terrell, G. Henkelman, Optimization methods for finding minimum energy paths, The Journal of Chemical Physics 128 (13) (2008). doi:10.1063/1.2841941.

[8] V. Asgeirsson, et al., Nudged elastic band method for molecular reactions using energy-weighted springs combined with eigenvector following (2021). doi:10.1021/acs.jctc.1c00462.

[9] G. Henkelman, H. Jonsson, Improved tangent estimate in the nudged elastic band method for finding minimum energy paths and saddle points (2000). doi:10.1063/1.1323224.

[10] S. Smidstrup, A. Pedersen, K. Stokbro, H. Jonsson, Improved initial guess for minimum energy path calculations (2014). doi:10.1063/1.4878664.

[11] W. E, W. Ren, E. Vanden-Eijnden, String method for the study of rare events (2002). doi:10.1103/PhysRevB.66.052301.

[12] W. E, W. Ren, E. Vanden-Eijnden, Simplified and improved string method for computing the minimum energy paths in barrier-crossing events (2007). doi:10.1063/1.2720838.

[13] R. A. Olsen, G. J. Kroes, G. Henkelman, et al., Comparison of methods for finding saddle points without knowledge of the final states (2004). doi:10.1063/1.1809574.

[14] J. Kastner, P. Sherwood, Superlinearly converging dimer method for transition state search (2008). doi:10.1063/1.2815812.

[15] R. Goswami, Reproducible orchestration of best practices for reaction path optimization with the nudged elastic band, MethodsX 16 (2026) 103899. doi:10.1016/j.mex.2026.103899.

[16] R. Goswami, A tutorial review of bayesian optimization with gaussian processes to accelerate stationary point searches (2026). doi:10.1021/acsphyschemau.6c00038.

[17] R. Goswami, M. Masterov, S. Kamath, A. Peña-Torres, H. Jónsson, Efficient implementation of Gaussian process regression accelerated saddle point searches with application to molecular reactions, Journal of Chemical Theory and Computation 21 (16) (2025) 7935–7943. doi:10.1021/acs.jctc.5c00866.

[18] R. Goswami, H. Jónsson, Adaptive pruning for increased robustness and reduced computational overhead in Gaussian process accelerated saddle point searches, ChemPhysChem 27 (4) (2026). doi:10.1002/cphc.202500730.

[19] O.-P. Koistinen, V. Asgeirsson, A. Vehtari, H. Jonsson, Nudged elastic band calculations accelerated with gaussian process regression based on inverse interatomic distances (2019). doi:10.1021/acs.jctc.9b00692.

[20] O.-P. Koistinen, V. Asgeirsson, A. Vehtari, H. Jonsson, Minimum mode saddle point searches using gaussian process regression with inverse-distance covariance function (2019). doi:10.1021/acs.jctc.9b01038.

[21] R. Goswami, M. Gunde, H. Jónsson, Enhanced climbing image nudged elastic band method with Hessian eigenmode alignment, Frontiers in Chemistry 14 (2026). doi:10.3389/fchem.2026.1807063.

[22] R. Goswami, Two-dimensional RMSD projections for reaction path visualization and validation, MethodsX 16 (2026) 103851. doi:10.1016/j.mex.2026.103851.

[23] S. T. Chill, M. Welborn, R. Terrell, L. Zhang, J.-C. Berthet, A. Pedersen, H. Jónsson, G. Henkelman, EON: software for long time simulations of atomic scale systems, Modelling and Simulation in Materials Science and Engineering 22 (5) (2014) 055002. doi:10.1088/0965-0393/22/5/055002.

[24] A. H. Larsen, J. J. Mortensen, J. Blomqvist, I. E. Castelli, R. Christensen, M. Dułak, J. Friis, M. N. Groves, B. Hammer, C. Hargus, E. D. Hermes, P. C. Jennings, P. B. Jensen, J. Kermode, J. R. Kitchin, E. L. Kolsbjerg, J. Kubal, K. Kaasbjerg, S. Lysgaard, J. B. Maronsson, T. Maxson, T. Olsen, L. Pastewka, A. Peterson, C. Rostgaard, J. Schiøtz, O. Schütt, M. Strange, K. S. Thygesen, T. Vegge, L. Vilhelmsen, M. Walter, Z. Zeng, K. W. Jacobsen, The atomic simulation environment—a Python library for working with atoms, Journal of Physics: Condensed Matter 29 (27) (2017) 273002. doi:10.1088/1361-648X/aa680e.

[25] P. de Buyl, P. H. Colberg, F. Höfling, H5MD: a structured, efficient, and portable file format for molecular data, Computer Physics Communications 185 (6) (2014) 1546–1553. doi:10.1016/j.cpc.2014.01.018.

[26] R. Rew, G. Davis, NetCDF: an interface for scientific data access, IEEE Computer Graphics and Applications 10 (4) (1990) 76–82. doi:10.1109/38.56302.

[27] M. Lundborg, R. Apostolov, D. Spångberg, A. Gärdenäs, D. van der Spoel, E. Lindahl, An efficient and extensible format, library, and API for binary trajectory data from molecular simulations, Journal of Computational Chemistry 35 (3) (2014) 260–269. doi:10.1002/jcc.23495.

[28] A. P. Thompson, H. M. Aktulga, R. Berger, D. S. Bolintineanu, W. M. Brown, P. S. Crozier, P. J. in 't Veld, A. Kohlmeyer, S. G. Moore,

T. D. Nguyen, R. Shan, M. J. Stevens, J. Tranchida, C. Trott, S. J. Plimpton, LAMMPS — a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales, Computer Physics Communications 271 (2022) 108171. doi:10.1016/j.cpc.2021.108171.

[29] M. Folk, G. Heber, Q. Koziol, et al., An overview of the hdf5 technology suite and its applications, 2011. doi:10.1145/1966895.1966900.

[30] Open Geospatial Consortium, Zarr storage specification 2.0 community standard. doi:10.62973/21-050r1.
URL https://doi.org/10.62973/21-050r1

[31] B. R. Brooks, et al., Charmm: The biomolecular simulation program (2009). doi:10.1002/jcc.21287.

[32] M. J. Abraham, et al., Gromacs: High performance molecular simulations through multi-level parallelism from laptops to supercomputers (2015). doi:10.1016/j.softx.2015.06.001.

[33] R. T. McGibbon, et al., Mdtraj: A modern open library for the analysis of molecular dynamics trajectories (2015). doi:10.1016/j.bpj.2015.08.015.

[34] H. M. Berman, The protein data bank (2000). doi:10.1093/nar/28.1.235.

[35] J. D. Westbrook, P. E. Bourne, Star/mmcif: An ontology for macromolecular structure (2000). doi:10.1093/bioinformatics/16/2/159.

[36] S. R. Hall, F. H. Allen, I. D. Brown, The crystallographic information file (cif): a new standard archive file for crystallography (1991). doi:10.1107/S010876739101067X.

[37] G. Kresse, J. Furthmuller, Efficiency of ab-initio total energy calculations for metals and semiconductors using a plane-wave basis set (1996). doi:10.1016/0927-0256(96)00008-0.

[38] S. P. Ong, et al., Python materials genomics (pymatgen): A robust, open-source python library for materials analysis (2013). doi:10.1016/j.commatsci.2012.10.028.

[39] P. Hirel, Atomsk: A tool for manipulating and converting atomic data files (2015). doi:10.1016/j.cpc.2015.07.012.

[40] N. M. O'Boyle, et al., Open babel: An open chemical toolbox (2011). doi:10.1186/1758-2946-3-33.

[41] A. Stukowski, Visualization and analysis of atomistic simulation data with ovito-the open visualization tool (2009). doi:10.1088/0965-0393/18/1/015012.

[42] V. Kapil, et al., i-pi 2.0: A universal force engine for advanced molecular simulations (2019). doi:10.1016/j.cpc.2018.09.020.

[43] M. Ceriotti, J. More, D. E. Manolopoulos, i-pi: A python interface for ab initio path integral molecular dynamics simulations (2014). doi:10.1016/j.cpc.2013.10.027.

[44] L. Martinez, R. Andrade, E. G. Birgin, et al., Packmol: A package for building initial configurations for molecular dynamics simulations (2009). doi:10.1002/jcc.21224.

[45] F. Bigi, et al., metatensor and metatomic: Foundational libraries for interoperable atomistic machine learning (2026). doi:10.1063/5.0304911.

[46] W. D. Clinger, How to read floating point numbers accurately, in: Proceedings of the ACM SIGPLAN 1990 Conference on Programming Language Design and Implementation, 1990, pp. 92–101. doi:10.1145/93542.93557.

[47] D. Lemire, Number parsing at a gigabyte per second, Software: Practice and Experience 51 (8) (2021) 1700–1727. doi:10.1002/spe.2984.

[48] G. L. Steele Jr., J. L. White, How to print floating-point numbers accurately, in: Proceedings of the ACM SIGPLAN 1990 Conference on Programming Language Design and Implementation, 1990, pp. 112–126. doi:10.1145/93542.93559.

[49] F. Loitsch, Printing floating-point numbers quickly and accurately with integers, 2010. doi:10.1145/1806596.1806623.

[50] U. Adams, Ryu: fast float-to-string conversion, 2018. doi:10.1145/3192366.3192369.

[51] G. Langdale, D. Lemire, Parsing gigabytes of json per second (2019). doi:10.1007/s00778-019-00578-5.

[52] G. Fraux, et al., chemfiles (2023). doi:10.5281/zenodo.1202207.
URL https://doi.org/10.5281/zenodo.1202207

[53] N. Michaud-Agrawal, E. J. Denning, T. B. Woolf, O. Beckstein, MDAnalysis: a toolkit for the analysis of molecular dynamics simulations, Journal of Computational Chemistry 32 (10) (2011) 2319–2327. doi:10.1002/jcc.21787.

[54] R. J. Gowers, M. Linke, J. Barnoud, T. J. E. Reddy, M. N. Melo, S. L. Seyler, J. Domański, D. L. Dotson, S. Buchoux, I. M. Kenney, O. Beckstein, MDAnalysis: a Python package for the rapid analysis of molecular dynamics simulations, in: Proceedings of the 15th Python in Science Conference, 2016, pp. 98–105. doi:10.25080/Majora-629e541a-00e.

[55] R. Goswami, readcon-core: a CON and convel reader and writer with a C interface (2026). doi:10.5281/zenodo.18753567.
URL https://doi.org/10.5281/zenodo.18753567

[56] R. Goswami, readcon-core: a con/convel reader/writer with ffi bindings (2026). doi:10.5281/zenodo.23264985.
URL https://doi.org/10.5281/zenodo.23264985

[57] M. Barker, et al., Introducing the fair principles for research software (2022). doi:10.1038/s41597-022-01710-x.

[58] A. M. Smith, D. S. Katz, K. E. Niemeyer, Software citation principles (2016). doi:10.7717/peerj-cs.86.

[59] R. Goswami, eon orchestrator (2026).
URL https://github.com/HaoZeke/eon_orchestrator